VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Faculty of Computer Science and Engineering

Textbook Exercises: Instruction Sets

Advanced System Architectures

CO5059

Instructor: Tran Ngoc Thinh

Notes by: Phan Nhat Minh (minh.phannhat95@hcmut.edu.vn)

Semester: 261

Hennessy & Patterson, Computer Architecture: A Quantitative Approach, *5th ed., Appendix A (Instruction Set Principles) exercises A.1–A.23. A map of every exercise with its page and relevance to Lecture 2, worked solutions for the ones our lectures cover, and hints for the rest.*

Last updated: October 4, 2026

Contents

1 How to Use This File **1**

2 Exercise Map **1**

3 Worked Solutions: High Priority **2**
Method for A.1–A.4 2
Exercise A.1 2
Exercise A.2 2
Exercise A.3 2
Exercise A.4 2
Exercise A.7 3
Exercise A.8 3
Exercise A.9 4
Exercise A.10 4
Exercise A.18 4
Exercise A.22 5

4 Worked Solutions: Medium Priority **5**
Exercise A.5 5
Exercise A.11 5
Exercise A.20 5
Exercise A.19, A.23 6

1 How to Use This File

- The exercise text is **not** copied here – open the book (`references/Computer Architecture - A Quantitative Approach.pdf`) at the page given. "Book p." is the printed page (Appendix A pages are numbered A-1, A-2, …); "PDF p." is the page in the PDF viewer.
- The book uses 64-bit MIPS (`LD`, `SD`, `DADD`, …); the ideas are the same as the 32-bit MIPS of our lecture.
- A.1–A.4 are the CPI-from-instruction-mix exercises (Lecture 1 skills applied to MIPS). They need Figures A.27 and A.28 (book pp. A-41 and A-42, PDF pp. 564–565).

2 Exercise Map

Ex.	Book p.	PDF p.	Topic	Priority
A.1	A-47	570	Effective CPI, SPECint mix (gap, gcc)	**High**
A.2	A-47	570	Effective CPI, SPECint mix (gzip, perlbmk)	**High**
A.3	A-47/48	570–571	Effective CPI with FP, SPECfp mix (lucas, swim)	**High**
A.4	A-48	571	Effective CPI with FP (applu, art)	**High**
A.5	A-48	571	Copy propagation on three statements	Medium
A.6	A-48	571	Lab: gcc optimization levels	Skip (lab)
A.7	A-48	571	MIPS and x86 code for an array loop; counts	**High**
A.8	A-48/49	571–572	Expanding-opcode encoding in 12 bits	**High**
A.9	A-49	572	Code size for the four ISA styles	**High**
A.10	A-49	572	Discussion: are more registers better?	**High**
A.11	A-50	573	Struct size and alignment, 32/64-bit	Medium
A.12–A.17	A-50/51	573–574	Labs: measuring mixes, AVX, gcc ports, energy	Skip (lab)
A.18	A-51/52	574–575	Memory traffic of the four ISA styles	**High**
A.19	A-52	575	ISA styles with register-indirect addressing (loop)	Medium
A.20	A-52/53	575–576	Choosing offset lengths from usage data	Medium
A.21	A-53/54	576–577	Lab: displacement histograms	Skip (lab)
A.22	A-54	577	Big/little endian, ASCII, misaligned reads	**High**
A.23	A-54	577	Discussion: ISA for desktop/server/cloud/embedded	Medium

3 Worked Solutions: High Priority

Method for A.1–A.4

Group the figure's rows into the CPI classes given in the exercise, average the two programs, then CPI $= \sum F_i \cdot \text{CPI}_i$. Conditional branches use $0.6 \times 2.0 + 0.4 \times 1.5 = 1.8$. Assumptions used below (state them in an exam): `jump`, `call` and `return` count as jumps; every other integer row (add, sub, mul, compare, load imm, shifts, logic, cond move) is an ALU instruction; `sub FP` counts as FP add; FP move/compare/cond-move/other count as "other FP".

Exercise A.1 (book p. A-47, PDF p. 570)

Asks: the effective CPI using the average of gap and gcc.

Solution. Loads+stores: gap 36.8%, gcc 38.3% → 37.55%. Cond. branches 10.7%. Jumps (jump+call+return): 4.0%, 1.9% → 2.95%. ALU: the rest, 48.8%.

$$\text{CPI} = 0.488(1.0)+0.3755(1.4)+0.107(1.8)+0.0295(1.2) = 0.488+0.526+0.193+0.035 \approx 1.24.$$

□

Exercise A.2 (book p. A-47, PDF p. 570)

Asks: the same with the average of gzip and perlbmk.

Solution. Loads+stores 35.05%, cond. branches 10.95%, jumps 2.75%, ALU 51.25%:

$$\text{CPI} = 0.5125 + 0.3505(1.4) + 0.1095(1.8) + 0.0275(1.2) \approx 1.23.$$

□

Exercise A.3 (book p. A-47/48, PDF p. 570–571)

Asks: the effective CPI including FP classes, average of lucas and swim (Fig. A.28).

Solution.

Class (CPI)	lucas	swim	avg	contribution
Integer ALU (1.0)	20.9	44.8	32.85	0.329
Int load/store (1.4)	14.0	10.4	12.2	0.171
Cond. branch (1.8)	0.6	1.3	0.95	0.017
FP load/store (1.5)	34.4	21.8	28.1	0.422
FP add/sub (4.0)	15.8	13.7	14.75	0.590
FP multiply (6.0)	9.4	6.9	8.15	0.489
FP divide (20.0)	0	0.3	0.15	0.030
Other FP (2.0)	5.0	0.9	2.95	0.059

CPI ≈ 2.11. FP arithmetic, although only ≈ 23% of the instructions, contributes over half of the cycles. □

Exercise A.4 (book p. A-48, PDF p. 571)

Asks: the same with the average of applu and art.

Solution.

Class (CPI)	applu	art	avg	contribution
Integer ALU (1.0)	50.4	42.1	46.25	0.463
Int load/store (1.4)	16.7	18.1	17.4	0.244
Cond. branch (1.8)	2.5	11.5	7.0	0.126
FP load/store (1.5)	15.6	16.5	16.05	0.241
FP add/sub (4.0)	5.2	4.5	4.85	0.194
FP multiply (6.0)	8.6	4.1	6.35	0.381
FP divide (20.0)	0.3	0.6	0.45	0.090
Other FP (2.0)	0.7	2.4	1.55	0.031

CPI ≈ 1.77. □

Exercise A.7 (book p. A-48, PDF p. 571)

Asks: code for `A[i] = B[i] + C` ($i = 0..100$, 64-bit data, everything kept in memory between uses) on MIPS and x86: dynamic instruction count, memory-data references and code size.

Solution. (a) One possible MIPS64 version (A, B, C, i at 1000, 3000, 5000, 7000):

```
        DADD  R1, R0, R0       ; i = 0
        SD    R1, 7000(R0)
Loop:   LD    R1, 7000(R0)     ; load i
        DSLL  R2, R1, #3       ; i * 8
        LD    R3, 3000(R2)     ; B[i]
        LD    R4, 5000(R0)     ; C
        DADD  R3, R3, R4
        SD    R3, 1000(R2)     ; A[i]
        DADDI R1, R1, #1       ; i++
        SD    R1, 7000(R0)     ; store i
        SLTI  R5, R1, #101     ; i <= 100 ?
        BNEZ  R5, Loop
```

Dynamic count $= 2 + 10 \times 101 = 1012$. Data references $= 1 + 5 \times 101 = 506$ (3 loads + 2 stores per iteration). Code size $= 12 \times 4 = 48$ bytes.

(b) x86 can use memory operands and scaled indexing, e.g. per iteration: `mov rax,[i]; mov rbx,[B+rax*8]; add rbx,[C]; mov [A+rax*8],rbx; add qword [i],1; cmp qword [i],100; jle Loop`. That is $1 + 7 \times 101 = 708$ instructions and $1 + 7 \times 101 = 708$ data references (the `add` to memory reads and writes). Code size depends on the encodings (variable length, roughly 4–10 bytes each): fewer instructions than MIPS, but more bytes per instruction and more memory traffic. □

Exercise A.8 (book p. A-48/49, PDF p. 571–572)

Asks: with 12-bit instructions and 5-bit register fields, can we encode given numbers of two-, one- and zero-address instructions?

Solution. Use expanding opcodes. Two-address: $12 - 10 = 2$ opcode bits, 4 patterns; 3 are used, 1 is left as an escape. One-address: the escape plus 5 more opcode bits gives $2^5 = 32$ patterns. Each one-address pattern left unused escapes to another 5 bits, i.e. 32 zero-address instructions.

(a) 3 / 30 / 45: 2 spare one-address patterns $\rightarrow 2 \times 32 = 64 \geq 45$. **Possible.**

(b) 3 / 31 / 35: 1 spare pattern $\rightarrow 32 < 35$. **Not possible.**

(c) 3 two-address and 24 zero-address need one escape pattern ($24 \leq 32$), so at most $32 - 1 = 31$ one-address instructions.

□

Exercise A.9 (book p. A-49, PDF p. 572)

Asks: (a) names per instruction and code size for $C = A + B$ on each ISA style of Fig. A.2 (8-bit opcode, 64-bit memory address, 6-bit register field); (b) a longer sequence, marking destroyed operands and overhead.

Solution. (a)

Style	Code (names per instruction)	Size
Stack	Push A (1), Push B (1), Add (0), Pop C (1)	$3 \cdot 72 + 8 = 224$ bits = 28 B
Accumulator	Load A, Add B, Store C (1 each)	$3 \cdot 72 = 216$ bits = 27 B
Reg-memory	Load R1,A; Add R3,R1,B; Store R3,C	$78 + 84 + 78 = 240$ bits = 30 B
Load-store	Load R1,A; Load R2,B; Add R3,R1,R2; Store R3,C	$78 + 78 + 26 + 78 = 260$ bits = 32.5 B

(b) Write each sequence the same way. Stack and accumulator machines destroy their operands (the top of stack / accumulator), so reusing A, C or D needs extra loads or pushes – those are the overhead instructions and overhead data bytes. Register machines keep A, C, D in registers and need no reloads, at the cost of longer instructions. □

Exercise A.10 (book p. A-49, PDF p. 572)

Asks: trade-offs of adding more registers to MIPS.

Solution. For: fewer loads/stores (less spilling), more values kept in registers across loops and calls, more room for compiler optimizations (register allocation, unrolling, scheduling). Against: more bits per register field (bigger instructions or fewer opcode and immediate bits), a larger and slower register file (more ports, longer access time, possibly a longer clock or extra pipeline stage), more state to save on context switches and calls, more power and area, and diminishing returns once the compiler cannot use them all. Changing the count also breaks binary compatibility. □

Exercise A.18 (book p. A-51/52, PDF p. 574–575)

Asks: code for `A=B+C; B=A+C; D=A-B` on accumulator, memory-memory, stack and load-store machines; mark reloads and operand passing; count instruction and data bytes for 16-bit and then 64-bit addresses/data.

Solution. Approach: 1-byte opcodes, direct addresses (2 bytes, then 8 bytes), register fields 4 bits (16 registers). Count, for each style, instruction bytes fetched and data bytes moved. Typical outcome: accumulator and stack code reload values (A, C) because their single storage place is overwritten; memory-memory has short programs but every operand is a memory access; load-store loads B, C once, keeps everything in registers and stores only

A, B, D. With 16-bit addresses the memory-oriented styles are competitive. With 64-bit addresses every address in an instruction costs 8 bytes, so load-store (which names memory least often) has the lowest total traffic. See our ISA review's practice problem on $D = (A + B) \times C$ for the same method. □

Exercise A.22 (book p. A-54, PDF p. 577)

Asks: store `434F 4D50 5554 4552` in an aligned double word, big and little endian, read it as ASCII, and list misaligned 2-byte (big-endian) and 4-byte (little-endian) reads.

Solution. The bytes `43 4F 4D 50 55 54 45 52` are the ASCII codes of `C O M P U T E R`.

(a) Big endian (address 0 to 7): `43 4F 4D 50 55 54 45 52` → "COMPUTER".

(b) Little endian: `52 45 54 55 50 4D 4F 43` → "RETUPMOC".

(c) Misaligned halfwords start at odd addresses 1, 3, 5: `0x4F4D`, `0x5055`, `0x5445`.

(d) Misaligned words that fit inside the double word start at addresses 1, 2, 3. In little endian the value is byte[a+3] byte[a+2] byte[a+1] byte[a]: `0x50555445`, `0x4D505554`, `0x4F4D5055`.

□

4 Worked Solutions: Medium Priority

Exercise A.5 (book p. A-48, PDF p. 571)

Asks: apply copy propagation to `A=B+C; B=A+C; D=A-B` until no operand is a computed value; where does work go up or down?

Solution. Using the original B and C: $A = B + C$; $B' = (B + C) + C = B + 2C$; $D = A - B' = (B + C) - (B + 2C) = -C$. Statement 2 got *more* work (two additions instead of one); statement 3 got much *less* (just a negation). So the same transformation can help or hurt: an optimizing compiler must evaluate costs case by case, which is why optimization is hard. □

Exercise A.11 (book p. A-50, PDF p. 573)

Asks: size of a given C struct on 32- and 64-bit machines, and the minimum size after reordering its members.

Solution. Assume each type is aligned to its own size (double 8, pointers 4 or 8). 32-bit: the members land at offsets 0, 1, 4, 8, 16, 20, 24, 32, 36, 40; the data ends at 44 and the struct is padded to a multiple of 8: **48 bytes**. Sorted largest first (doubles, then 4-byte items, short, chars): **40 bytes**. 64-bit (8-byte pointers): offsets 0, 1, 4, 8, 16, 20, 24, 32, 40, 48; ends at 52, padded to **56 bytes**. Reordered: **48 bytes**. □

Exercise A.20 (book p. A-52/53, PDF p. 575–576)

Asks: average instruction length if offsets may be 0, 8, 16 or 24 bits, versus a fixed 24-bit instruction (with extra instructions for long offsets) or always-24-bit offsets.

Solution. Method: an n-bit offset including sign holds $n-1$ magnitude bits, so read the cumulative percentages of Fig. A.31 at 0, 7, 15 and 23 bits (data: 30.4%, 66.9%, 100%; branches: 0.1%, 85.2%, 98.5%, 100%). Weight each offset length by the load/store and branch frequencies of Fig. A.27 (ALU and other instructions are 16 bits), and compare the bytes fetched. The point of the exercise: variable-length offsets save bytes on average, but a fixed format is simpler to decode, which is the trade-off behind MIPS's fixed 16-bit immediates. □

Exercise A.19, A.23 (book p. A-52, A-54, PDF p. 575, 577)

Solution. A.19: same method as A.7 and A.18, writing the loop for each ISA style; register-indirect addressing lets the stack/accumulator machines walk the arrays with a pointer instead of absolute addresses. A.23: desktop/server favour performance and compatibility (rich ISAs, 64-bit, SIMD); cloud/warehouse favours throughput per watt and virtualization support; embedded favours code density, low power and cost (compressed encodings such as Thumb/MIPS16, small register files). □